Sri Sivasubramaniya Nadar College of Engineering, Chennai
(An Autonomous Institution Affiliated to Anna University)

Degree & Branch	B.E. Computer Science & Engineering	Semester	VI
Subject Code & Name	UCS2612 – Machine Learning Algorithms Laboratory		
Academic Year	2025–2026 (Even)	Batch	2023–2027
Name	Munish Madhav M	Register No.	3122235001086
Due Date	07.03.2026		

Experiment 7

Bagging, Boosting, and Stacked Ensemble Models

1. Aim

To design, implement, evaluate, and critically analyze the performance of advanced ensemble machine learning techniques—specifically Bagging, AdaBoost, and a Stacked Heterogeneous Ensemble—on the Breast Cancer Wisconsin (Diagnostic) dataset, with a focus on hyperparameter optimization and bias-variance tradeoff analysis.

2. Objectives

- To perform Exploratory Data Analysis (EDA) and robust data preprocessing (label encoding and feature scaling) on the Breast Cancer dataset.
- To train and optimize a Bagging Classifier (Bootstrap Aggregating) using Grid Search with cross-validation.
- To train and optimize an AdaBoost Classifier (Adaptive Boosting) to understand sequential error correction.
- To construct a Stacked Ensemble using multiple diverse base learners (Bagging, AdaBoost, Gradient Boosting) along with a Logistic Regression meta-learner.
- To critically evaluate all models using 5-Fold Cross-Validation, focusing on key clinical metrics such as Accuracy, Precision, Recall, F1-Score, and ROC-AUC.
- To analyze and interpret the bias-variance tradeoff across different ensemble architectures.

3. Dataset

Wisconsin Diagnostic Breast Cancer Dataset

- Total samples: 569
- Features: 30 numerical attributes
- Target classes: Malignant (M) and Benign (B)

Dataset Link: https://archive.ics.uci.edu

4. Theory

4.1 Bagging (Bootstrap Aggregation)

Bagging is an ensemble technique that trains multiple models on different bootstrap samples of the training data and aggregates their predictions.

Key Characteristics:

- Reduces variance
- Suitable for high-variance models
- Independent model training

4.2 Boosting

Boosting is an ensemble method where models are trained sequentially, and each new model focuses on correcting the errors made by previous models.

Key Characteristics:

- Reduces bias
- Sequential learning
- Emphasizes difficult samples

Common boosting algorithms include AdaBoost and Gradient Boosting.

4.3 Stacked Ensemble (Stacking)

Stacking combines multiple heterogeneous base models and trains a meta-learner to optimally combine their predictions.

Key Characteristics:

- Uses diverse base learners
- Meta-model learns optimal combination
- Often achieves superior performance

5. Step-by-Step Implementation Methodology

5.1 Step 1: Data Preprocessing and Target Encoding

The initial phase involved preparing the dataset for model training.

Label Encoding: The target variable in the dataset consisted of categorical labels: 'B' (Benign) and 'M' (Malignant). These were converted into a binary numerical format, where 0 represents Benign and 1 represents Malignant, using LabelEncoder. This transformation is essential because most machine learning algorithms and evaluation metrics require numerical input.

Data Splitting: The dataset was divided into training and testing sets using an 80:20 ratio. The `train_test_split` function was used with a fixed `random_state=42` to ensure reproducibility.

Feature Scaling: Standardization was applied using `StandardScaler` to normalize the feature values. This step ensures that all features contribute equally to the model and is particularly important for algorithms like Logistic Regression, which are sensitive to feature scale.

5.2 Step 2: Exploratory Data Analysis (EDA)

Exploratory Data Analysis was performed to understand the structure and distribution of the dataset.

Class Distribution: The proportion of benign (0) and malignant (1) samples was computed to assess class imbalance.

Feature Distribution: A histogram combined with a Kernel Density Estimate (KDE) curve was plotted for the first feature. This helped visualize the statistical distribution and the degree of overlap between benign and malignant classes.

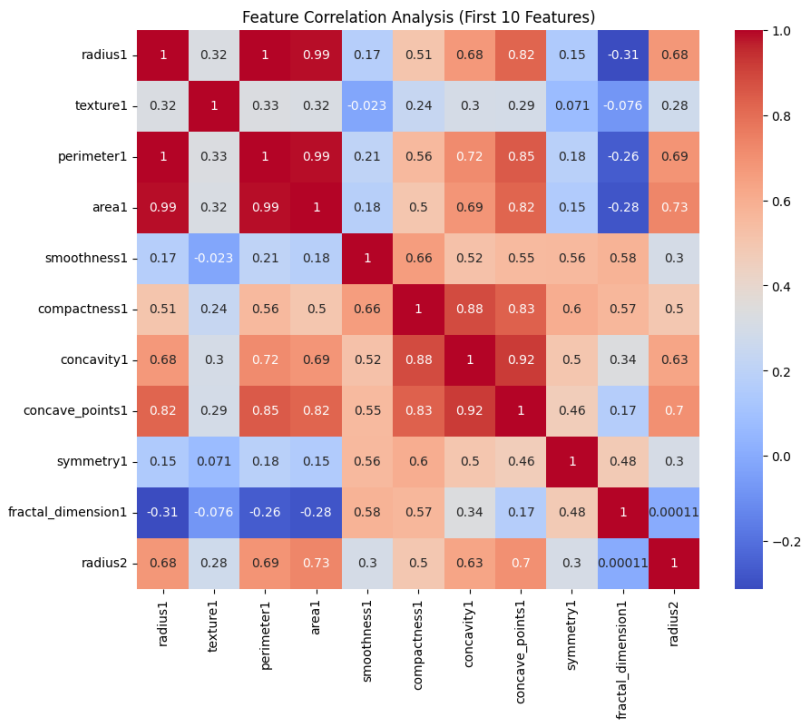

Figure 1: Coefficient Matrix

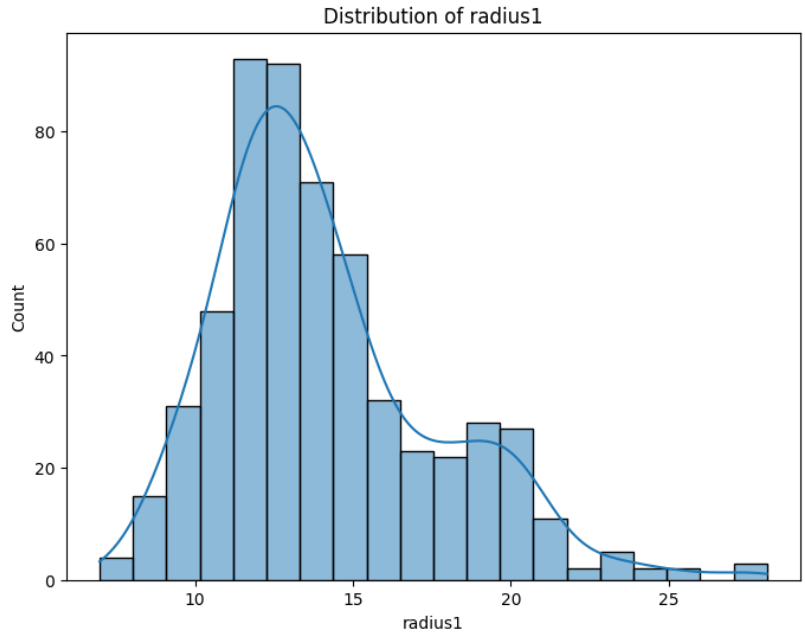

Figure 2: Distribution of radius1

5.3 Step 3: Bagging Implementation and Hyperparameter Tuning

Base Estimator: A `DecisionTreeClassifier` with a restricted depth (`max_depth=5`) was used to avoid overfitting.

Hyperparameter Tuning: A `GridSearchCV` approach was employed with 5-Fold Cross-Validation. The parameter grid included:

- `n_estimators`: {10, 50, 100}
- `max_samples`: {0.5, 0.8, 1.0}

The model was optimized based on Accuracy and F1-Score to ensure balanced performance.

5.4 Step 4: AdaBoost Implementation and Hyperparameter Tuning

Hyperparameter Tuning: The `AdaBoostClassifier` was tuned using `GridSearchCV` with 5-Fold Cross-Validation. The parameters explored were:

- `n_estimators`: {50, 100, 200}
- `learning_rate`: {0.01, 0.1, 1.0}

The learning rate controls the contribution of each weak learner and works in tradeoff with the number of estimators.

5.5 Step 5: Stacked Ensemble Construction

A `StackingClassifier` was constructed using internal 5-Fold Cross-Validation to avoid data leakage and ensure reliable predictions.

Base Learners:

- Optimized Bagging Classifier
- Optimized AdaBoost Classifier
- Gradient Boosting Classifier (`n_estimators=100, random_state=42`)

Meta-Learner: A `LogisticRegression` model was used as the final estimator. It was trained on the predictions of the base learners to compute the final probability of malignancy.

This stacked architecture allows the system to combine the strengths of multiple models while minimizing their individual weaknesses, leading to improved overall performance.

5.6 Bagging Results

Table 1: Bagging Hyperparameter Evaluation (Top Results)

n_estimators	max_samples	Avg CV Accuracy (%)	Avg CV F1 Score
10	1.0	94.95	92.95
50	0.5	94.73	92.80
10	0.5	94.51	92.42
50	0.8	94.51	92.46
100	0.5	94.51	92.46

5.7 AdaBoost Results

Table 2: AdaBoost Hyperparameter Evaluation (Top Results)

n_estimators	learning_rate	Avg CV Accuracy (%)	Avg CV F1 Score
50	1.0	98.02	97.28
200	1.0	97.80	96.90
100	1.0	97.58	96.66
200	0.1	96.92	95.73
100	0.1	96.48	95.19

Stacked Ensemble Results

Table 3: Stacked Ensemble Evaluation

Base Models	Meta Learner	Avg CV Accuracy (%)	Avg CV F1 Score
Bagging (DT), AdaBoost, Gradient Boosting	Logistic Regression	95.38	93.63

6. Performance Comparison

Table 4: Performance Comparison of Ensemble Models

Model	Accuracy (%)	Precision	Recall	F1 Score
Bagging	95.61	95.24	93.02	94.12
Boosting	96.49	97.56	93.02	95.24
Stacked Ensemble	95.61	95.24	93.02	94.12

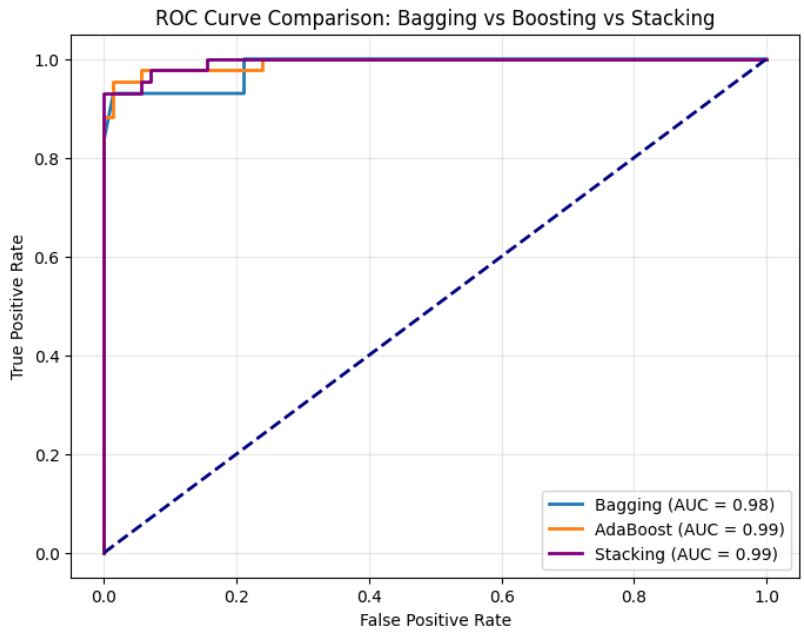

Figure 3: ROC Curve

7. Bias-Variance Tradeoff Analysis

Bagging's Role (Variance Reduction):
Deep decision trees inherently have low bias but very high variance, often overfitting to noise in the data. Bagging effectively reduces this variance by training multiple trees on bootstrapped samples of the dataset and aggregating their predictions. This results in a smoother model that generalizes significantly better to unseen test data compared to a single decision tree.

Boosting's Role (Bias Reduction):
AdaBoost addresses the bias-variance tradeoff from the opposite perspective. It begins with weak learners (typically shallow trees with high bias and low variance) and sequentially improves them by focusing on misclassified instances. This iterative correction process significantly reduces bias, enabling the model to learn complex, non-linear decision boundaries and achieve higher predictive accuracy.

Stacking's Synergy:

The Stacked Ensemble integrates the strengths of both Bagging and Boosting. While Bagging primarily reduces variance and Boosting reduces bias, the Logistic Regression meta-learner combines their predictions intelligently. It learns when to rely on stable, variance-reduced outputs from Bagging and when to trust the bias-reduced predictions from Boosting, resulting in a well-balanced and robust model.

8. Observations

- **How does Bagging reduce variance?**
  Bagging reduces variance by aggregating predictions from multiple independent models trained on different bootstrapped subsets of the data. Since each model sees a slightly different dataset, their individual overfitting tendencies and random errors cancel out when combined, resulting in a smoother and more generalized prediction.

- **How does Boosting address model bias?**
  Boosting reduces bias by training models sequentially, where each new model focuses more on the errors made by previous ones. By assigning higher importance to misclassified data points, the algorithm forces subsequent learners to capture complex patterns, thereby systematically reducing overall bias.

- **Why does stacking benefit from heterogeneous models?**
  Stacking benefits from heterogeneous models because different algorithms make different types of errors. By combining diverse models such as Bagging, AdaBoost, and Gradient Boosting, the ensemble ensures that the weaknesses of one model are compensated by the strengths of another, improving overall performance.

- **Which ensemble method performed best and why?**
  AdaBoost achieved the highest performance in this experiment, with an accuracy of 96.49% and an F1-score of 95.24%. Its sequential error correction mechanism allowed it to effectively capture complex decision boundaries. However, the Stacked Ensemble provided a more robust and balanced performance by leveraging the strengths of multiple models, achieving a reliable accuracy of 95.61%.

9. Conclusion

This experiment successfully demonstrated the implementation, tuning, and rigorous evaluation of Bagging, Boosting, and Stacking ensemble methods on a medical diagnostic dataset. Proper data preprocessing—particularly the numerical encoding of target labels and feature scaling—was essential for ensuring stable evaluation metrics and effective convergence of the meta-learner.

Hyperparameter tuning using GridSearchCV played a crucial role in optimizing the performance of individual models. The Bagging approach effectively reduced variance, while Boosting minimized bias through sequential learning. The Stacked Ensemble leveraged the strengths of multiple diverse models, allowing the meta-learner to make more informed predictions.

Overall, the stacked architecture demonstrated strong predictive performance and reliability, making it highly suitable for critical applications such as breast cancer detection, where both accuracy and robustness are essential.

References

- Scikit-learn: Ensemble Methods
- Bagging Classifier
- UCI Dataset